Option / Maybe

Season: 6 · FP Foundations · Category: Functional · Level: Intermediate

Intent

- Model absence without null.

Problem

- Without this pattern, code tends toward: New things up with conditionals and duplicated construction.

Real-world Analogy

- A sealed box that is Some or None.

When to Use

- Pure functional style in Java 25 + Zazr.
- The design force named in Intent shows up more than once.

When Not to Use

- Throwing exceptions everywhere is 'fine' for a script.
- A simpler language feature (record, sealed switch, function value) already solves it.

Pattern Structure

- See mermaid diagram in explanation.md

Classic Implementation

```java
package com.buildwithwaqar.patterns.functional.option;

import dev.zazr.control.Option;

import static dev.zazr.control.Option.none;
import static dev.zazr.control.Option.some;

/** Option / Maybe with Zazr — sealed Some/None, pattern-matchable. */
public final class ZazrOptionDemo {
    private ZazrOptionDemo() {}

    public static Option<Integer> parsePositive(String raw) {
        try {
            int n = Integer.parseInt(raw);
            return n > 0 ? some(n) : none();
        } catch (NumberFormatException e) {
            return none();
        }
    }

    public static String describe(Option<Integer> opt) {
        return switch (opt) {
            case Option.Some(var n) -> "got " + n;
            case Option.None() -> "missing";
        };
    }

    public static Option<Integer> sum3(Option<Integer> a, Option<Integer> b, Option<Integer> c) {
        return Option.zipWith(a, b, c, (x, y, z) -> x + y + z);
    }
```

Classic Implementation (2)

```
}
```

Modern Java 25 / Functional

- Primary examples use Zazr; VAVR shown in comparison episode.

Worked Example

```
See the JUnit test beside the implementation.
Run: mvn -Dtest=*Test test
Focus package matching: functional/option/
```

Pitfalls

- Zazr rejects null inside Option/collections.
- Keep interpreters at the edge.

Interview Q&A

- Q: Interview: what problem does Option / Maybe solve? A: Model absence without null.
- Q: How would you test this? A: Prefer injecting interfaces or interpreters; keep demos deterministic (fixed clocks, canned inputs).

Key Takeaways

- Model absence without null.
- Prefer Java 25 sealed types / records / virtual threads where they simplify the classic shape.
- Ship working tests with every episode.

Video Script Highlights

- [0:00] Hook — Open with the analogy: A sealed box that is Some or None. [0:40] Intent — State the design force in one sentence: Model abs...